Práctica 4
CreatorNet: Red de Colaboración de Creadores/as Digitales

Algoritmos y Estructuras de Datos - Grado en Ingeniería Informática (USC)

18 de octubre de 2026

Objetivo de la versión 1

Adaptar un TAD grafo para representar una red no dirigida y valorada, cargar una red inicial desde un fichero y permitir su consulta y modificación.

Contexto

La plataforma CreatorVerse quiere poner en marcha CreatorNet, una aplicación para gestionar las colaboraciones posibles entre sus creadores/as de contenido. La red se representará mediante un grafo: cada creador/a será un vértice y cada colaboración posible entre dos creadores/as será un arco. Las colaboraciones pueden realizarse de forma directa, mediante una producción conjunta, o de forma remota, integrando posteriormente las aportaciones de las dos personas participantes. Cada colaboración tendrá un coste de producción y una duración estimada.

Fase 1: Estudio del proyecto base

Antes de desarrollar la aplicación CreatorNet, deberás comprender el funcionamiento de un TAD grafo ya implementado.

Se proporciona un proyecto de ejemplo que implementa un **grafo no dirigido y no valorado cuyos vértices son números enteros**. Este proyecto constituye una referencia de uso del TAD y deberás estudiarlo antes de realizar cualquier modificación. El proyecto proporcionado incluye los siguientes archivos:

- `main.c`
- `grafo.c`
- `grafo.h`
- `misfunciones.c`
- `misfunciones.h`

Tareas

1. Compilar y ejecutar el proyecto.
2. Analizar la estructura general de la aplicación.
3. Estudiar la interfaz pública definida en `grafo.h`.
4. Comprender la representación interna utilizada en `grafo.c`.
5. Identificar cómo se implementan las operaciones básicas sobre el grafo.
6. Analizar las funciones auxiliares contenidas en `misfunciones.h` y `misfunciones.c`, identificando cuáles podrían servir como referencia para el desarrollo de CreatorNet.

Objetivos de aprendizaje

Al finalizar esta fase, deberás ser capaz de:

- Describir la representación utilizada para los vértices y los arcos.
- Comprender cómo se insertan, eliminan y consultan elementos del grafo.
- Reutilizar la estructura proporcionada en una aplicación diferente.

Fase 2: Desarrollo de CreatorNet

El objetivo de esta fase es desarrollar una aplicación denominada **CreatorNet** para gestionar una red de colaboraciones entre creadores/as de contenido utilizando un **grafo no dirigido y valorado**.

Material proporcionado

Para el desarrollo de esta fase se proporcionarán los siguientes archivos encapsulados en `CreatorNet.zip`:

- `main.c`
- `creatornet.h`
- `creatornet.c`
- `creatornet_version1.csv`

Los archivos `creatornet.h` y `creatornet.c` contienen las funciones necesarias para inicializar la aplicación, comprobar los argumentos de la línea de comandos y leer el fichero de datos. También se proporciona el esqueleto de las funciones encargadas de procesar los registros de creadores y colaboraciones. Deberás completar únicamente las partes señaladas en estas funciones para incorporar correctamente los datos al grafo.

Organización del proyecto

La solución final deberá estar organizada exclusivamente mediante los siguientes archivos:

- `main.c`: programa principal y gestión del menú,
- `grafo.h`: interfaz pública del TAD grafo,
- `grafo.c`: representación interna e implementación del TAD grafo,

- `creatornet.h`: interfaz de las funciones específicas de CreatorNet, y
- `creatornet.c`: inicialización, carga de datos y funciones específicas de la aplicación.

Los archivos `misfunciones.h` y `misfunciones.c` pertenecen únicamente al proyecto de referencia de la Fase 1 y no deberán incluirse en la solución final.

Preparación del nuevo proyecto

Antes de implementar las funcionalidades de CreatorNet, deberás crear un nuevo proyecto a partir de la estructura analizada en la Fase 1.

Podrás copiar en este nuevo proyecto los archivos `grafo.h` y `grafo.c` del proyecto de referencia. A partir de ellos deberás desarrollar un nuevo TAD grafo adaptado a los requisitos de CreatorNet.

Se proporciona un archivo `main.c` con la estructura básica de la aplicación, aunque podrás utilizar como referencia la estructura general del archivo `main.c` de la aplicación de ejemplo de la Fase 1 y las funciones contenidas en `misfunciones.h` y `misfunciones.c`. Las funciones que necesites para el nuevo proyecto deberás copiarlas en los archivos correspondientes de CreatorNet: los prototipos en `creatornet.h` y las implementaciones en `creatornet.c`. **Una vez copiadas, deberás modificarlas en estos nuevos archivos** para adaptarlas a los tipos de datos y a las necesidades de CreatorNet.

Importante: No deberás modificar `misfunciones.h` ni `misfunciones.c`, ya que pertenecen al proyecto de referencia de la Fase 1 y no formarán parte de la solución final.

Antes de modificar el TAD, deberás identificar:

- qué aspectos de la implementación proporcionada pueden mantenerse,
- qué tipos de datos deben modificarse,
- qué operaciones deben adaptarse para trabajar con los nuevos vértices y arcos,
- qué funciones del proyecto de referencia pueden reutilizarse o servir como modelo.

Representación de la red

CreatorNet representará la red de posibles colaboraciones mediante un grafo no dirigido y valorado. Los vértices representarán a los/as creadores/as y los arcos, las posibles colaboraciones entre dos vértices.

Vértices

Cada vértice representará un/a creador/a de contenido. Para simplificar el modelo utilizado en la práctica de árboles, deberás almacenar únicamente la siguiente información de cada creador/a:

- `alias`: identificador único de cada creador/a.
- `categoria`: categoría principal. Las categorías disponibles serán: 1) streamer, 2) artista digital o musical, 3) jugador/a de e-sports, 4) podcaster o divulgador/a tecnológico/a y 5) desarrollador/a de videojuegos. Se representará mediante un valor entero comprendido entre 1 y 5.
- `colectivo`: colectivo, equipo o canal al que pertenece, representado mediante una cadena de caracteres. Cuando el/la creador/a no pertenezca a ningún colectivo se utilizará la cadena "-".

Para definir las longitudes de los arrays utilizados para almacenar el alias y el colectivo, deberán emplearse constantes simbólicas denominadas obligatoriamente `TAM_ALIAS` y `TAM_COLECTIVO`. Deberás determinar los valores adecuados para ambas constantes teniendo en cuenta los datos que se van a almacenar y la necesidad de reservar espacio para el carácter de terminación `'\0'`.

Estas constantes deberán utilizarse de forma coherente en todas las declaraciones y operaciones relacionadas con dichos campos. No se utilizarán directamente valores numéricos para indicar las longitudes de estos arrays.

A partir de estos requisitos, deberás modificar la definición de `tipovertice` proporcionada en el proyecto de referencia para adaptarla a la información almacenada en CreatorNet.

Arcos

Cada arco representará una posible colaboración entre dos creadores/as. Para cada colaboración se almacenará la siguiente información:

- Coste de producción, expresado mediante un valor real no negativo.
- Duración estimada, expresada mediante un valor real positivo.
- Modalidad de colaboración, que podrá tomar uno de los siguientes valores: `D` cuando sea una colaboración directa o `R` cuando sea una colaboración remota.

La matriz de adyacencia almacenará elementos del tipo definido para representar los arcos. Puesto que CreatorNet utiliza un **grafo no dirigido**, cualquier inserción, modificación o eliminación de una colaboración deberá actualizar de forma coherente las dos posiciones simétricas de la matriz de adyacencia.

Adaptación del TAD grafo

Antes de implementar la carga inicial y las funcionalidades de CreatorNet, deberás adaptar el TAD estudiado en la Fase 1 a las características de la nueva red. Para ello será necesario:

1. Adaptar la definición de los vértices para almacenar la información de los/as creadores/as.
2. Definir un tipo adecuado para representar los datos asociados a las colaboraciones.
3. Modificar las operaciones necesarias para trabajar con los nuevos tipos de datos.
4. Realizar la comparación entre vértices exclusivamente mediante el alias.
5. Adaptar las operaciones sobre arcos para gestionar una red valorada.
6. Mantener la simetría de la matriz de adyacencia en todas las operaciones sobre colaboraciones.
7. Mantener encapsulada la representación interna del grafo en `grafo.c`.

Para ello deberás modificar las definiciones de `TIPOVERTICE` y `TIPOARCO`, manteniendo la estructura general del TAD basada en `TGRAFO`.

Inicialización y carga de la red

El programa se ejecutará indicando el fichero de datos mediante la opción `-f`:

```
./creatornet -f creatornet_version1.csv
```

La función `main()` llamará a una función de inicialización proporcionada, que se encargará de crear el grafo, comprobar los argumentos de la línea de comandos e intentar cargar los datos iniciales.

Si los argumentos no son correctos o el fichero no puede abrirse, se mostrará un mensaje informativo. Estas circunstancias no provocarán la finalización del programa, que continuará ejecutándose con una red vacía. Si se detecta un registro incorrecto, se mostrará un mensaje informativo y se continuará procesando el resto del fichero. Los registros cargados correctamente permanecerán incorporados al grafo.

El fichero contiene dos tipos de registros:

```
C;alias;categoria;colectivo
A;alias_origen;alias_destino;coste;duracion;modalidad
```

Los registros que comienzan por `C` describen creadores/as y aparecerán antes que los registros que comienzan por `A`, que describen colaboraciones.

Cada colaboración aparecerá una sola vez en el fichero. Puesto que CreatorNet se representa mediante un grafo no dirigido, la operación del TAD utilizada para insertar una colaboración deberá actualizar de forma coherente las dos posiciones simétricas de la matriz de adyacencia.

En `creatornet.c` se proporcionarán implementadas las funciones encargadas de:

- inicializar el grafo,
- comprobar los argumentos,
- abrir y cerrar el fichero,
- leerlo línea a línea,
- identificar el tipo de cada registro y
- gestionar los errores generales de apertura y lectura.

También se proporcionará la estructura de las funciones privadas `_procesarCreador()` y $_procesarColaboracion()destinadasaprocesaruncreadoryunacolaboracin, respectivamente, contodaslasvari$

`Deberás completar únicamente estas dos funciones para:`

- `validar los campos de cada registro,`
- `comprobar que los campos de texto respetan las longitudes definidas,`
- `convertir los datos numéricos,`
- `construir los vértices y los valores asociados a los arcos,`
- `comprobar la existencia de los elementos referenciados y detectar posibles duplicados e`
- `incorporar los datos mediante las operaciones públicas del TAD grafo.`

`No deberás modificar el código proporcionado para la inicialización, la comprobación de argumentos ni la lectura general del fichero.`

Funcionalidades de CreatorNet

La aplicación deberá proporcionar, al menos, las siguientes funcionalidades:

A. Anhadir creador/a
E. Eliminar creador/a
C. Crear colaboracion
B. Eliminar colaboracion
I. Mostrar creadores/as y colaboraciones
L. Consultar colaboraciones de un/a creador/a
S. Salir

Añadir creador/a

Permitirá incorporar un/a nuevo/a creador/a. Se solicitarán alias, categoría y colectivo. No se realizará la inserción si ya existe un/a creador/a con ese alias, si la categoría no está entre 1 y 5 o si se ha alcanzado el número máximo de vértices.

Eliminar creador/a

Permitirá eliminar un/a creador/a existente. Se solicitará el alias. Si existe, se eliminarán el vértice y todas sus colaboraciones. Si no existe, se mostrará un mensaje informativo.

Crear colaboración

Permitirá la creación de una nueva colaboración entre dos creadores/as. Se solicitarán los dos alias, el coste, la duración y la modalidad. La operación se rechazará cuando:

- alguno de los alias no corresponda a un vértice existente,
- los dos alias identifiquen el mismo vértice,
- ya exista una colaboración entre ambos vértices,
- el coste sea negativo,
- la duración sea menor o igual que cero o
- la modalidad no sea D ni R

Eliminar colaboración

Permitirá la eliminación de una colaboración existente. Se solicitarán los dos alias. Si ambos alias corresponden a vértices existentes y entre ellos hay una colaboración, se eliminará el arco en los dos sentidos. En cualquier otro caso se mostrará un mensaje adecuado.

Mostrar creadores/as y colaboraciones

Se mostrará cada creador/a y la lista de sus colaboraciones, incluyendo alias de destino, modalidad, coste y duración. Por ejemplo:

```
AdaLoop (categoria 4, NightOwls)
  [R] CodeComet   coste: 140.00 €  duracion: 8.00 h
  [D] CosmicMoth  coste: 260.00 €  duracion: 5.50 h
```

La modalidad de cada colaboración se representará mediante [D] para las colaboraciones directas y [R] para las colaboraciones remotas. En la visualización completa de la red no será necesario evitar que cada arco aparezca dos veces, pues se mostrará la lista de adyacencias de cada vértice.

Consultar colaboraciones de un/a creador/a

La aplicación permitirá consultar las colaboraciones asociadas a un/a creador/a determinado/a. Se solicitará el alias que se desea consultar. Si corresponde a un vértice existente, se mostrarán todas sus colaboraciones, indicando para cada una el alias del otro extremo, la modalidad, el coste y la duración. Se utilizará el mismo formato que en la visualización completa de la red: [D] para las colaboraciones directas y [R] para las colaboraciones remotas. Por ejemplo:

```
Alias que se desea consultar: AdaLoop

Colaboraciones de AdaLoop:
  [R] CodeComet:  coste 140.00 €  duracion 8.00 h
  [D] CosmicMoth: coste 260.00 €  duracion 5.50 h
```

Requisitos de implementación

- El código deberá compilar sin warnings con las opciones indicadas en el guion general de la práctica.
- No se permitirán variables globales que almacenen el grafo ni matrices de adyacencia auxiliares.
- La representación interna del TAD grafo deberá permanecer encapsulada en grafo.c.
- Las cadenas deberán copiarse de forma segura y respetar las longitudes definidas mediante TAM_ALIAS y TAM_COLECTIVO.
- Todas las entradas deberán validarse.
- Los errores no deberán provocar la finalización abrupta del programa.
- La memoria dinámica utilizada deberá liberarse antes de finalizar la ejecución.
- Cada función deberá tener una responsabilidad clara. No se concentrará toda la lógica de la aplicación en main.c.
- Las funciones de creatornet.c deberán utilizar exclusivamente la interfaz pública del TAD para consultar o modificar el grafo.

Pruebas mínimas

Además de comprobar la carga inicial, deberás realizar y documentar al menos las siguientes pruebas:

- Carga correcta del fichero inicial.
- Ejecución con un fichero inexistente y continuación con una red vacía.
- Inserción correcta de un/a creador/a.
- Intento de inserción de un alias duplicado.
- Intento de inserción de una categoría no válida.
- Creación correcta de una colaboración.
- Intento de crear una colaboración entre un/a creador/a y sí mismo/a.
- Intento de creación de una colaboración ya existente.
- Intento de creación de una colaboración con una modalidad no válida.
- Eliminación de una colaboración.
- Eliminación de un/a creador/a con colaboraciones asociadas.
- Verificación de la simetría y consistencia del grafo tras las operaciones anteriores.